```
PRAGMA foreign_keys = ON;

CREATE TABLE Authors (
    author_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
);

CREATE TABLE Books (
    book_id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    genre TEXT NOT NULL,
    publisher TEXT,
    year INTEGER
);

CREATE TABLE BookAuthors (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    author_id INTEGER NOT NULL,
    FOREIGN KEY (book_id) REFERENCES
Books(book_id) ON DELETE CASCADE,
    FOREIGN KEY (author_id) REFERENCES
Authors(author_id) ON DELETE CASCADE
);
```

```
CREATE TABLE Members (
    member_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    phone TEXT,
    address TEXT
);

CREATE TABLE Copies (
    copy_id INTEGER PRIMARY KEY AUTOINCREMENT,
    book_id INTEGER NOT NULL,
    status TEXT NOT NULL DEFAULT 'Available',
    FOREIGN KEY (book_id) REFERENCES
Books(book_id) ON DELETE CASCADE
);

CREATE TABLE Transactions (
    transaction_id INTEGER PRIMARY KEY
AUTOINCREMENT,
    member_id INTEGER NOT NULL,
    copy_id INTEGER NOT NULL,
    issue_date DATE NOT NULL,
    return_date DATE,
```

```
    FOREIGN KEY (member_id) REFERENCES
Members(member_id),
    FOREIGN KEY (copy_id) REFERENCES
Copies(copy_id)
);

CREATE TABLE Reservations (
    reservation_id INTEGER PRIMARY KEY
AUTOINCREMENT,
    member_id INTEGER NOT NULL,
    book_id INTEGER NOT NULL,
    reservation_date DATE NOT NULL,
    FOREIGN KEY (member_id) REFERENCES
Members(member_id),
    FOREIGN KEY (book_id) REFERENCES
Books(book_id)
);

CREATE TABLE Fines (
    fine_id INTEGER PRIMARY KEY AUTOINCREMENT,
    transaction_id INTEGER NOT NULL,
    amount REAL NOT NULL,
    status TEXT NOT NULL CHECK(status IN
('Paid','Unpaid')),
```

```
    FOREIGN KEY (transaction_id) REFERENCES
Transactions(transaction_id)
);

-- INDEXES )

CREATE INDEX idx_book_title ON Books(title);
CREATE INDEX idx_member_name ON Members(name);

-- TRIGGERS

-- Issue book -> status = Issued
CREATE TRIGGER issue_book
AFTER INSERT ON Transactions
BEGIN
    UPDATE Copies
    SET status = 'Issued'
    WHERE copy_id = NEW.copy_id;
END;

-- Return book -> status = Available

CREATE TRIGGER return_book
AFTER UPDATE ON Transactions
```

```sql
WHEN NEW.return_date IS NOT NULL
BEGIN
    UPDATE Copies
    SET status = 'Available'
    WHERE copy_id = NEW.copy_id;
END;

-- VIEW

CREATE VIEW issued_books AS
SELECT
    m.name AS member_name,
    b.title AS book_title,
    t.issue_date
FROM Transactions t
JOIN Members m ON t.member_id = m.member_id
JOIN Copies c ON t.copy_id = c.copy_id
JOIN Books b ON c.book_id = b.book_id
WHERE t.return_date IS NULL;
```